

Experiment No.	2
Experiment Title	Email Spam Classification using Naive Bayes and K-Nearest Neighbors ¹
Student Name	Samyuktha V
Register Number	3122247001053
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	12/08/26

1 Objective

To implement and compare Gaussian, Multinomial, and Bernoulli Naive Bayes along with K-Nearest Neighbors (KNN) for email spam classification, tune KNN's hyperparameters using GridSearchCV and RandomizedSearchCV, compare KDTree vs. BallTree search algorithms, evaluate all models using 5-fold cross-validation, and compare them on accuracy, precision, recall, F1, ROC-AUC, and training/prediction time.

2 Problem Statement

Given the preprocessed Spambase dataset (57 numeric word/character-frequency features, output of Experiment 1's `assn1_preprocess()`), classify each email as spam (1) or ham (0). Input: scaled feature vector per email. Output: binary spam/ham prediction.

3 Dataset Description

Dataset Name	Spambase (updated), preprocessed
Dataset Source	UCI Machine Learning Repository (Spam-base)
Number of Samples	3,968 (3,174 train / 794 test)
Number of Features	57 (MinMax scaled, 0–1 range)
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	0 (already cleaned in Experiment 1)
Train-Test Split	80% / 20%, stratified, <code>random_state=42</code> (carried over from Experiment 1)

4 Exploratory Data Analysis

Full EDA (class distribution, feature histograms, boxplots, correlation heatmap) was performed in the Experiment 1 report, since the same `assn1_preprocess()` output feeds directly into this experiment.

5 Data Preprocessing

Inherited directly from Experiment 1 – no additional preprocessing was performed here, except a Bernoulli-specific step:

- **Bernoulli NB feature binarization:** since BernoulliNB expects 0/1 "presence/absence" features rather than continuous frequencies, all features were binarized using a threshold of 0.5 (`Binarizer(threshold=0.5)`) before training that specific model only.

6 Machine Learning Algorithm

Gaussian Naive Bayes

Working principle: assumes each feature, within each class, follows a normal distribution.

Mathematical formulation: $P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i-\mu_y)^2}{2\sigma_y^2}\right)$, combined via Bayes' theorem.

Advantages: fast, works with continuous features, no hyperparameters.

Limitations: the normality assumption is a poor fit for skewed word-frequency data (confirmed in Experiment 1's EDA), which limits its accuracy here.

Multinomial Naive Bayes

Working principle: models features as counts drawn from a multinomial distribution – suited to word-frequency/count data.

Mathematical formulation: $P(x|y) \propto \prod_i P(w_i|y)^{x_i}$.

Advantages: natural fit for Bag-of-Words / count features; fast; no hyperparameters.

Limitations: requires non-negative feature values; ignores feature correlations (the "naive" independence assumption).

Bernoulli Naive Bayes

Working principle: models each feature as a binary presence/absence indicator.

Mathematical formulation: $P(x_i|y) = p_{iy}^{x_i} (1 - p_{iy})^{1-x_i}$.

Advantages: explicitly penalizes the *absence* of a feature, unlike Multinomial NB.

Limitations: discards frequency information by binarizing – a poor fit when *how often* a word appears carries useful signal, as seen in this dataset's results below.

K-Nearest Neighbors (KNN)

Working principle: classifies a point by majority vote among its k nearest neighbours in feature space.

Mathematical formulation: distance typically Euclidean, $d(x, x') = \sqrt{\sum_i (x_i - x'_i)^2}$; predicted class = $\text{mode}(y_i : x_i \in N_k(x))$.

Advantages: simple, no training phase, naturally handles multi-class problems.

Limitations: slow prediction on large datasets (brute-force is $O(nd)$ per query); sensitive to feature scale (hence MinMax scaling in Experiment 1) and to the choice of k .

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.9.0
IDE	Jupyter Notebook
Operating System	Windows
Hardware	Intel i5, 16GB RAM

8 Hyperparameter Selection

Parameter	Search Space
KNN k (manual sweep)	1, 3, 5, 7, 9, 11
KNN <code>n_neighbors</code> (Grid/RandomSearch)	1–19
KNN <code>weights</code>	uniform, distance
Cross-Validation Folds	5
Search Methods	GridSearchCV, RandomizedSearchCV (n_iter=10)

Search Method	Best Parameters	CV Accuracy
GridSearchCV	<code>n_neighbors=11, weights='distance'</code>	0.9112
RandomizedSearchCV	<code>n_neighbors=8, weights='distance'</code>	0.9124

Best k from manual sweep (Section 5, $k = 1,3,5,7,9,11$): $k = 7$ (test accuracy 0.9018)

9 Results

KNN Accuracy Across Manual k Sweep

k	Accuracy
1	0.8955
3	0.8816
5	0.9005
7	0.9018
9	0.8992
11	0.8955

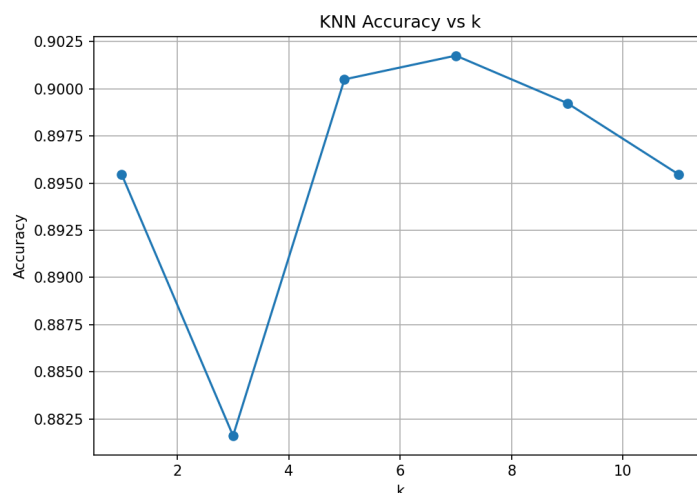


Figure 1: KNN Accuracy vs. k

5-Fold Cross-Validation (Training Set)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Gaussian NB	0.8189	0.8535	0.8252	0.8441	0.8297	0.8343
Best KNN (k=7)	0.9008	0.9102	0.8819	0.8961	0.8943	0.8967

KDTree vs. BallTree (k=7)

Algorithm	Accuracy	Precision	Recall	F1	Train Time (s)	Predict Time (s)
KDTree	0.9018	0.9188	0.8423	0.8789	0.0111	0.1606
BallTree	0.9018	0.9188	0.8423	0.8789	0.0071	0.1432

Final Model Comparison (Test Set)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train Time (s)
Multinomial NB	0.9131	0.9185	0.8720	0.8947	0.9623	0.0021
Best KNN (k=7)	0.9018	0.9188	0.8423	0.8789	0.9539	0.0010
KNN (BallTree)	0.9018	0.9188	0.8423	0.8789	0.9539	0.0071
KNN (KDTree)	0.9018	0.9188	0.8423	0.8789	0.9539	0.0111
Gaussian NB	0.8514	0.7583	0.9524	0.8443	0.9052	0.0034
Bernoulli NB	0.6058	0.8966	0.0774	0.1425	0.5811	0.0029

10 Result Visualization

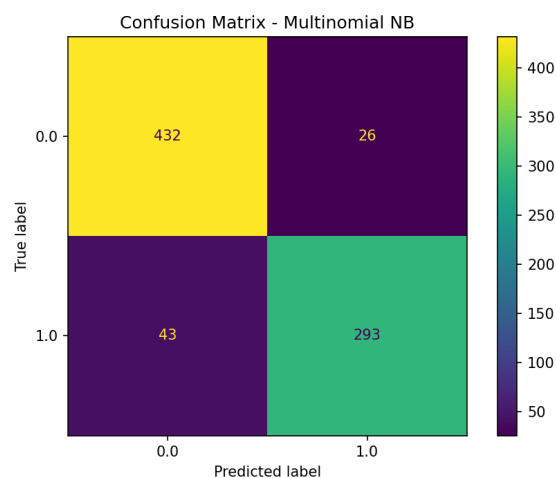


Figure 2: Confusion Matrix – Multinomial Naive Bayes (best overall model)

Inference:

Multinomial NB correctly classifies the large majority of both spam and ham emails, with relatively few false negatives (spam mislabelled as ham). This matches expectations: word-count features align naturally with the multinomial distribution’s assumptions, giving it an edge over the other Naive Bayes variants.

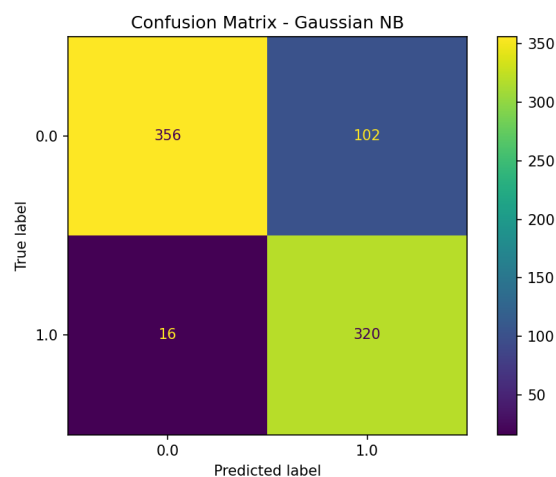


Figure 3: Confusion Matrix – Gaussian Naive Bayes

Inference:

Gaussian NB catches spam well (recall = 0.9524, the highest of any model) but at the cost of many false positives – legitimate ham emails misclassified as spam (precision only 0.7583). This is consistent with the EDA finding that word-frequency features are far from normally distributed, which biases Gaussian NB’s probability estimates.

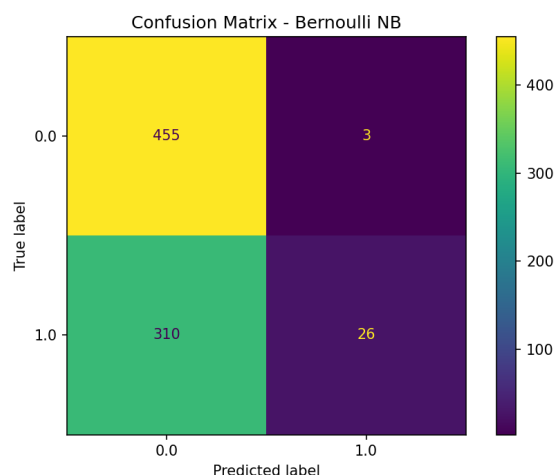


Figure 4: Confusion Matrix – Bernoulli Naive Bayes

Inference:

Bernoulli NB performs poorly here (accuracy 0.6058, recall only 0.0774) – almost all spam emails are missed. Binarizing continuous word-frequency features at a threshold of 0.5 throws away most of the signal, since the vast majority of word-frequency values in this dataset are well below 0.5 after MinMax scaling and collapse to 0 regardless of their original relative size.

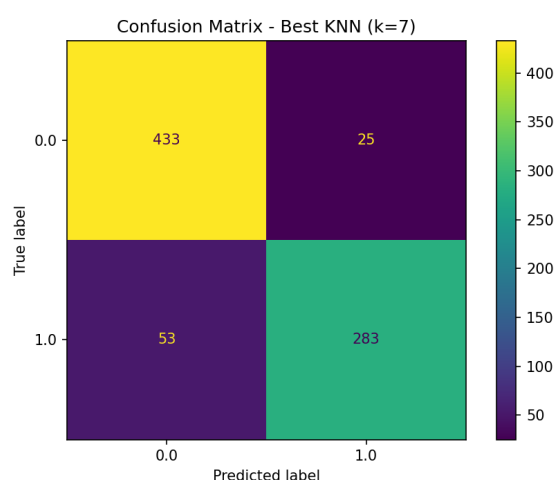


Figure 5: Confusion Matrix – Best KNN (k=7)

Inference:

KNN with k=7 performs strongly and consistently across accuracy, precision, and F1, though its recall (0.8423) trails Multinomial NB's (0.8720) – it misses slightly more spam emails. KDTree and BallTree produce *identical* accuracy/ precision/recall (as expected, since both are exact nearest-neighbour search methods, differing only in internal data structure), but BallTree was faster to train and predict on this dataset.

11 Discussion

Multinomial Naive Bayes was the clear winner on this dataset (accuracy 0.9131, ROC-AUC 0.9623), confirming the EDA-based prediction from Experiment 1 that a count-based Naive Bayes variant would outperform Gaussian NB on skewed word-frequency data. Bernoulli NB was by far the weakest model, since a single binarization threshold (0.5) is a poor match for MinMax-scaled word-frequency data where most values cluster near 0. KNN performed competitively (0.9018 accuracy at $k=7$) and its manual sweep, GridSearchCV, and RandomizedSearchCV all converged to similar k values (7, 11, and 8 respectively) with CV accuracies within about half a percentage point of each other – suggesting the model’s performance is fairly stable across that range of k rather than being highly sensitive to the exact choice. KDTree and BallTree gave identical predictive results (both are exact search algorithms), with BallTree slightly faster on this dataset’s 57-dimensional feature space.

12 Conclusion

Across all 6 models compared, **Multinomial Naive Bayes** gave the best overall test performance (Accuracy 0.9131, F1 0.8947, ROC-AUC 0.9623) while also being the fastest to train, making it the recommended model for this spam classification task. KNN ($k=7$) was a close second and is a reasonable alternative if a probability-free, distance-based approach is preferred, though it is slower at prediction time than Naive Bayes. Bernoulli NB’s binarization strategy is not well suited to this dataset’s already-scaled continuous features and should be avoided here.

13 References

1. Scikit-learn: Naive Bayes, https://scikit-learn.org/stable/modules/naive_bayes.html
2. Scikit-learn: Nearest Neighbors, <https://scikit-learn.org/stable/modules/neighbors.html>
3. UCI Machine Learning Repository: Spambase Dataset